

Chapter 12

Language Models for Information Retrieval

Complete Lecture Notes | Topics 12.1 through 12.2.2
Includes Student Questions and Professor Answers

Section	Topic	Key Concept
12.1 Intro	Big Picture	$P(\text{query} \text{doc model})$ vs $P(\text{Relevant} \text{doc}, \text{query})$
12.1.1	Finite Automata and LMs	Probability distributions over strings
12.1.2	Types of Language Models	Unigram vs Bigram — why IR uses Unigram
12.1.3	Multinomial Distributions	Bag-of-words; query multinomial coefficient cancels
12.2.1	Query Likelihood Model	Rank docs by $P(q M_d)$
12.2.2	Estimating Query Probability	MLE + Smoothing (Jelinek-Mercer and Bayesian)

Topic 1 — Section 12.1: Introduction and Big Picture

The Central Idea

Before Chapter 12, the book taught **Probabilistic IR (Chapter 11)**, which asks:

Probabilistic IR (Chapter 11):

$P(\text{Relevant} \mid \text{document, query})$ — Is this document relevant to the query?

Chapter 12 introduces a completely different way of thinking. Instead of modeling relevance directly, we ask:

Language Modeling Approach (Chapter 12):

$P(\text{query} \mid \text{document model})$ — Would this document naturally generate my query?

The Human Intuition

The Librarian's Tip (from the book)

"Think of words that would likely appear in a relevant document and use those as your query."

The Language Modeling approach formalizes this tip mathematically. We build a language model M_d from each document d , and rank documents by $P(q \mid M_d)$ — the probability that the document's model generates the query.

Comparison: Three IR Approaches

Feature	Vector Space Model	Probabilistic IR (Ch.11)	Language Model (Ch.12)
Core idea	Cosine similarity of vectors	$P(\text{Relevant} \mid \text{doc, query})$	$P(\text{query} \mid \text{document model})$
What it models	Geometric distance	Relevance probability	Query generation probability
Explicit relevance?	No	Yes	No
Foundation	Linear algebra	Probability theory	Probabilistic language theory
Term weights	tf-idf (heuristic)	Log odds ratios	MLE + smoothing

Topic 2 — Section 12.1.1: Finite Automata and Language Models

Part A: What is a Finite Automaton?

A **finite automaton** is a machine that generates strings of words by following fixed rules. Example (Figure 12.1 from the book) — it can generate:

"I wish"

"I wish I wish"

"I wish I wish I wish"

...and so on

But it **cannot** generate 'wish I wish' — the order is fixed. This is still deterministic — no probabilities yet.

Part B: Adding Probabilities — Language Model

Now imagine each state has a probability distribution over all words in the vocabulary (Figure 12.2 from the book):

Word	Probability
the	0.20
a	0.10
frog	0.01
toad	0.01
said	0.03
likes	0.02
that	0.04
...	...

All probabilities sum to 1. After generating each word, the model either **stops** or **loops back** to generate another word. This is a **Language Model**.

Part C: Formal Definition

Formal definition of a Language Model:

Sum over all strings s in alphabet* of $P(s) = 1$

In plain English: the probabilities of ALL possible strings must sum to 1. It is a proper probability distribution over text.

Worked Example — P(string) Calculation

EXAM TIP

This exact type of calculation is frequently asked in exams! Practice it until you can do it without notes.

Given: $P(\text{frog})=0.01$, $P(\text{said})=0.03$, $P(\text{that})=0.04$, $P(\text{toad})=0.01$, $P(\text{likes})=0.02$

Stop probability = 0.2, **Continue probability** = 0.8

Query string: 'frog said that toad likes frog' (6 words)

Step	Component	Values	Result
1	Word probabilities (multiply all 6)	$0.01 \times 0.03 \times 0.04 \times 0.01 \times 0.02 \times 0.01$	2.4×10^{-12}
2	Continue 6 times (0.8 each)	$0.8 \times 0.8 \times 0.8 \times 0.8 \times 0.8 \times 0.8$	0.2621
3	Stop once at end	0.2	0.2
4	Combine steps 2 and 3	0.2621×0.2	0.05243
5	Final P(string)	$2.4 \times 10^{-12} \times 0.05243$	approx 1.258×10^{-13}

Example 12.2 — Comparing Two Language Models (Likelihood Ratio)

Word	$P(\text{word} \mid M1)$	$P(\text{word} \mid M2)$
frog	0.01	0.0002
said	0.03	0.03
that	0.04	0.04
toad	0.01	0.0001
likes	0.02	0.04
dog	0.005	0.01

Query string s: 'frog said that toad likes that dog'

$P(s \mid M1)$:

$0.01 \times 0.03 \times 0.04 \times 0.01 \times 0.02 \times 0.04 \times 0.005 = \text{approx } 4.8 \times 10^{-13}$

$P(s \mid M2)$:

$0.0002 \times 0.03 \times 0.04 \times 0.0001 \times 0.04 \times 0.04 \times 0.01 = \text{approx } 3.84 \times 10^{-16}$

Likelihood Ratio:

$$P(s|M1) / P(s|M2) = 4.8 \times 10^{-13} / 3.84 \times 10^{-16} = 1250$$

Conclusion: M1 is 1250 times more likely to have generated this query. Document d1 ranks higher.

Key Observations

Observation	Explanation
Probabilities are very tiny	This is completely normal. Always compare them, never interpret absolute values.
Stop probabilities can be omitted when ranking	Stop probability is constant across models for same query. It cancels out in the likelihood ratio and does not affect ranking.
Likelihood ratio is the key metric	The model giving higher P(query) wins. Ranking by probability = ranking by likelihood ratio.

COMMON STUDENT MISTAKE

Mistake 1: Forgetting to multiply the stop probability — it is needed formally but OK to omit when ranking. Mistake 2: Panicking at tiny probabilities — always compare them, never interpret absolute values. Mistake 3: Dividing instead of multiplying word probabilities — always MULTIPLY.

Topic 3 — Section 12.1.2: Types of Language Models

The Core Problem and the Chain Rule

How do we assign probability to a sequence like 'the cat sat on the mat'? The chain rule gives the mathematically exact answer:

Chain Rule:

$$P(t_1 t_2 t_3 t_4) = P(t_1) \times P(t_2|t_1) \times P(t_3|t_1, t_2) \times P(t_4|t_1, t_2, t_3)$$

Mathematically perfect but impossibly complex for long sequences. So we simplify.

Type 1: Unigram Language Model (Used in IR)

Idea: throw away ALL context. Every word is independent of every other word.

Unigram Language Model:

$$P_{\text{uni}}(t_1 t_2 t_3 t_4) = P(t_1) \times P(t_2) \times P(t_3) \times P(t_4)$$

Example: $P(\text{'the cat sat'}) = P(\text{the}) \times P(\text{cat}) \times P(\text{sat}) = 0.2 \times 0.01 \times 0.02 = 0.00004$

Type 2: Bigram Language Model

Idea: each word depends only on the one previous word.

Bigram Language Model:

$$P_{\text{bi}}(t_1 t_2 t_3 t_4) = P(t_1) \times P(t_2|t_1) \times P(t_3|t_2) \times P(t_4|t_3)$$

Why Does IR Use ONLY Unigram Models?

EXAM TIP

Classic exam question — write ALL three reasons for full marks.

Reason	Explanation
1. Topic judgment does not need word order	To know a document is about 'climate change', you just need those words to appear frequently. Word order is irrelevant for topic detection.
2. Data sparseness (Bias-Variance Tradeoff)	A language model in IR is estimated from a SINGLE document (maybe 500 words). Not enough data to reliably estimate bigram probabilities. With limited data, a simpler (more constrained) model performs better.
3. Efficiency	Unigram models are faster to estimate and apply. In a collection of millions of documents, this matters enormously.

Model Comparison Table

Model	Depends On	Formula Style	Used in IR?
Unigram	Nothing (independent)	$P(t_1) \times P(t_2) \times P(t_3)$	YES
Bigram	Previous 1 word	$P(t_1) \times P(t_2 t_1) \times P(t_3 t_2)$	Rarely
Trigram	Previous 2 words	$P(t_1) \times P(t_2 t_1) \times P(t_3 t_1,t_2)$	No

Topic 4 — Section 12.1.3: Multinomial Distributions over Words

The Key Insight

Since we use unigram models, word order does not matter. The document becomes a 'bag of words' — a collection of words with counts, no order. A **Multinomial Distribution** gives us the probability of a bag of words.

The Full Multinomial Formula

Multinomial probability of document d:

$$P(d) = [L_d! / (tf_{t1,d}! \times tf_{t2,d}! \times \dots \times tf_{tM,d}!)] \times P(t_1)^{tf_{t1,d}} \times P(t_2)^{tf_{t2,d}} \times \dots$$

Symbol	Meaning	Example
L_d	Total number of word tokens in document d	Doc = 'the cat the' → $L_d = 3$
$tf_{t_i,d}$	How many times word t_i appears in document d	$tf_{the,d} = 2$, $tf_{cat,d} = 1$
M	Total vocabulary size (number of unique words)	$M = 50,000$
$P(t_i)$	Probability of word t_i in the language model	$P(the) = 0.2$
$L_d! / (tf_{t1,d}! \times tf_{t2,d}! \times \dots)$	Multinomial coefficient — number of orderings of the same bag	For 'the the cat': $3! / (2! \times 1!) = 3$

Worked Example

Document d: 'the cat the cat the' — $L_d = 5$, $tf_{the} = 3$, $tf_{cat} = 2$

Given: $P(the) = 0.2$, $P(cat) = 0.01$

Step	Calculation	Result
Multinomial coefficient	$5! / (3! \times 2!) = 120 / (6 \times 2)$	10
Word probability product	$(0.2)^3 \times (0.01)^2 = 0.008 \times 0.0001$	0.0000008
Final $P(d)$	10×0.0000008	0.000008

The Critical Shortcut for IR

EXAM TIP

The multinomial coefficient of the QUERY (not the document) is constant across all documents because the same query is used for all. It cancels out when ranking. We can ignore it completely. Important: L_d is NOT fixed — it differs per document. It is L_q (query length) that is fixed.

Simplified ranking formula (query multinomial coefficient dropped):

$P(d)$ is proportional to Product over t in V of $P(t)^{tf_{t,d}}$

STUDENT QUESTION

Q: You said 'the multinomial coefficient is identical for both' — does that mean L_d is fixed?

A: Great question — this needed clarification. L_d (document length) is NOT fixed across documents; it differs per document. What IS fixed is L_q (query length), because the same query is used for all documents. The multinomial coefficient that cancels out is the one for the QUERY (which depends on L_q), not the one for the document (which depends on L_d). So L_d is different per document, but the query coefficient is constant and cancels out during ranking.

Clarification Table: Fixed vs Not Fixed

Quantity	Fixed Across Documents?	Why?
L_d (document length)	NO	Different documents have different lengths
L_q (query length)	YES	Same query is used for all documents being compared
Multinomial coefficient of the document	NO	Depends on L_d — varies per document
Multinomial coefficient of the query	YES	Depends on L_q — constant, cancels out in ranking

COMMON STUDENT MISTAKE

Common student question: 'If word order does not matter, why call it a language model?' Answer: The name comes from the ORIGINS of the technique (speech recognition, NLP). In IR we borrow the tool and apply it to ranking. The 'language' part is historical.

Topic 5 — Section 12.2.1: The Query Likelihood Model

What is the Query Likelihood Model?

Core Idea

For each document d , build a language model M_d .

Rank documents by $P(q | M_d)$ — the probability of the query being generated by the document's language model.

Documents that would naturally 'produce' your query words rank highest.

Derivation Using Bayes Rule

Bayes Rule applied to document ranking:

$$P(d | q) = P(q | d) \times P(d) / P(q)$$

Term	Action	Reason
$P(q)$	IGNORE (drop it)	$P(q)$ is the same for ALL documents for a given query — it is constant and cancels out
$P(d)$	Treat as UNIFORM (equal for all d)	We assume all documents are equally likely a priori. Could implement genuine prior based on authority, length, newness, etc.
$P(q d)$	This is what we COMPUTE	The probability the document's language model generates the query — this distinguishes documents

Final ranking criterion:

Rank documents by: $P(q | M_d)$

The Three-Step Process

Step	Action	Description
1	Build language model M_d	Infer a language model for each document from its word frequencies
2	Compute $P(q M_d)$	Estimate the probability of generating the query according to each document's model
3	Rank documents	Sort documents in decreasing order of $P(q M_d)$

Comparison: LM vs Probabilistic IR vs VSM

Aspect	Vector Space Model	Probabilistic IR (BIM)	Query Likelihood LM
Models	Similarity between vectors	$P(\text{Relevant} \mid \text{doc}, \text{query})$	$P(\text{query} \mid \text{document model})$
Relevance variable	Not explicit	Central variable	Not modeled explicitly
Term weighting	tf-idf (heuristic)	Log odds ratio c_t	MLE + smoothing
Collection stats	Used heuristically (idf)	Used in BIM priors	Integral part of model
Relevance feedback	Rocchio algorithm	Re-estimate p_t and u_t	Harder without explicit relevance
Performance	Good baseline	Good with BM25	Very strong — beats tf-idf

Topic 6 — Section 12.2.2: Estimating the Query Generation Probability

NOTE: This is the most numerically important section of Chapter 12. It covers MLE estimation and two smoothing methods — all are exam favourites.

Step 1 — Maximum Likelihood Estimation (MLE)

MLE estimate of $P(t | M_d)$:

$$\hat{P}_{\text{mle}}(t | M_d) = \text{tf}_{t,d} / L_d$$

Symbol	Meaning
$\hat{P}_{\text{mle}}(t M_d)$	MLE estimate of probability of term t under document d 's language model
$\text{tf}_{t,d}$	Raw term frequency — how many times term t appears in document d
L_d	Total number of tokens in document d (document length)

MLE query probability (unigram, multiply over all query terms):

$$\hat{P}(q | M_d) = \text{Product over } t \text{ in } q \text{ of } (\text{tf}_{t,d} / L_d)$$

The Problem with Pure MLE — Zero Probabilities

If a query term does not appear in a document, $\text{tf}_{t,d} = 0$, so:

$$\hat{P}_{\text{mle}}(t | M_d) = 0 / L_d = 0$$

Since we MULTIPLY probabilities for all query terms, the entire document gets score ZERO — even if it contains all other query terms. This is called **strict conjunctive semantics**.

Problem	Consequence
Missing query terms get $P = 0$	Entire document score becomes 0 — any number multiplied by 0 equals 0
Words occurring once are overestimated	Their one occurrence may be by chance — probability is inflated by MLE
Unseen words get 0 probability	Zero is both an underestimate and breaks the entire model

EXAM TIP

IMPORTANT: The role of smoothing in LMs for IR is NOT simply to avoid zero probabilities. Smoothing actually implements major parts of the TERM WEIGHTING component. An unsmoothed model works badly because it lacks term weighting — not just because of zeros. This distinction is very commonly asked in exams.

The Solution — Smoothing

Smoothing simultaneously does three things:

Decreases the estimated probability of words we DID see (to avoid overestimation)

Increases the probability of words we DID NOT see (to avoid zero probabilities)

Implements term weighting — rare collection words appearing in a document get higher relative weight

Constraint on smoothed probability for unseen terms:

If $tf_{t,d} = 0$ then $P_{\hat{t}}(t | M_d)$ is less than or equal to cf_t / T

Symbol	Meaning
cf_t	Raw count of term t in the ENTIRE collection
T	Total number of tokens in the entire collection
cf_t / T	Probability of term t by chance from the whole collection — the upper bound for unseen terms

Smoothing Method 1 — Jelinek-Mercer (Linear Interpolation)

Jelinek-Mercer Smoothing (Equation 12.10 in book):

$P_{\hat{t}}(t | d) = \lambda \times P_{\hat{t}}_{mle}(t | M_d) + (1 - \lambda) \times P_{\hat{t}}_{mle}(t | M_c)$

Symbol	Meaning	Value Range
λ	Mixing weight — controls how much we trust the document vs the collection	$0 < \lambda < 1$
$P_{\hat{t}}_{mle}(t M_d)$	MLE from document: $tf_{t,d} / L_d$	0 to 1
$P_{\hat{t}}_{mle}(t M_c)$	MLE from collection: cf_t / T (probability of t in whole collection)	0 to 1
$(1 - \lambda)$	Weight given to the collection model	$0 < (1 - \lambda) < 1$

Value of λ	Effect	Best For
Small λ (e.g. 0.1)	More smoothing — heavily relies on collection model	Long queries
Large λ (e.g. 0.9)	Less smoothing — mainly uses document model	Short queries
$\lambda = 0.5$	Balanced combination — used in book's Example 12.3	General use, default in examples

Smoothing Method 2 — Bayesian Smoothing (Dirichlet)

Bayesian Smoothing (Equation 12.11 in book):

$$P_{\text{hat}}(t | d) = (tf_{t,d} + \alpha \times P_{\text{hat}}(t | M_c)) / (L_d + \alpha)$$

Symbol	Meaning
α	Smoothing strength — larger α = more smoothing = more weight on collection
$tf_{t,d}$	Raw term frequency of t in document d
L_d	Document length (total tokens)
$P_{\text{hat}}(t M_c)$	Collection model probability of term t

Comparison of the Two Smoothing Methods

Feature	Jelinek-Mercer (Linear Interpolation)	Bayesian Smoothing
Formula	$\lambda \times P_{\text{mle}}(t d) + (1-\lambda) \times P_{\text{mle}}(t M_c)$	$(tf_{t,d} + \alpha \times P(t M_c)) / (L_d + \alpha)$
Parameter	λ ($0 < \lambda < 1$)	α (positive number)
More smoothing when	λ is smaller	α is larger
Less smoothing when	λ is larger	α is smaller
Book reference	Equation (12.10)	Equation (12.11)
Also called	Jelinek-Mercer smoothing	Bayesian or Dirichlet smoothing

Final ranking formula (Jelinek-Mercer smoothing):

$$P(d|q) \text{ proportional to } P(d) \times \text{Product over } t \text{ in } q \text{ of } [(1-\lambda) \times P(t|M_c) + \lambda \times P(t|M_d)]$$

Worked Example 12.3 — Full Numerical Calculation

EXAM TIP

This is THE classic exam numerical problem for Chapter 12. Work through every step. The format here is exactly what exams ask.

Problem Setup

Document collection (2 documents):

d1: 'Xyzyzy reports a profit but revenue is down' (8 words)

d2: 'Quorus narrows quarter loss but revenue decreases further' (8 words)

Query: 'revenue down'

Model: MLE unigram mixed with collection model at $\lambda = 1/2$

Assumption: $P(d)$ is uniform — ignore the prior

Step 1 — Build the Collection Model (16 total words)

Term	Count in d1	Count in d2	Total Count	$P(t Mc) = \text{count} / 16$
revenue	1	1	2	$2/16 = 1/8 = 0.125$
down	1	0	1	$1/16 = 0.0625$
but	1	1	2	$2/16 = 1/8 = 0.125$
profit	1	0	1	$1/16$
loss	0	1	1	$1/16$
(all others)	1 each	varies	1 each	$1/16$ each

Step 2 — Build Document Models (MLE) (each doc has 8 words)

Term	tf in d1	$P_{\text{mle}}(t d1) = \text{tf}/8$	tf in d2	$P_{\text{mle}}(t d2) = \text{tf}/8$
revenue	1	$1/8 = 0.125$	1	$1/8 = 0.125$
down	1	$1/8 = 0.125$	0	$0/8 = 0$

Step 3 — Apply Smoothing ($\lambda = 0.5$)

Smoothed $P(t | d) = 0.5 \times P_{\text{mle}}(t | d) + 0.5 \times P(t | Mc)$

For document d1:

Term	$0.5 \times P_{\text{mle}}(t d1)$	$0.5 \times P(t Mc)$	Smoothed $P(t d1)$
revenue	$0.5 \times 1/8 = 1/16$	$0.5 \times 1/8 = 1/16$	$1/16 + 1/16 = 2/16 = 1/8$
down	$0.5 \times 1/8 = 1/16$	$0.5 \times 1/16 = 1/32$	$1/16 + 1/32 = 2/32 + 1/32 = 3/32$

For document d2:

Term	$0.5 \times P_{\text{mle}}(t d2)$	$0.5 \times P(t Mc)$	Smoothed $P(t d2)$
revenue	$0.5 \times 1/8 = 1/16$	$0.5 \times 1/8 = 1/16$	$1/16 + 1/16 = 1/8$

Term	$0.5 \times P_{\text{mle}}(t d_2)$	$0.5 \times P(t Mc)$	Smoothed $P(t d_2)$
down	$0.5 \times 0 = 0$	$0.5 \times 1/16 = 1/32$	$0 + 1/32 = 1/32$

Step 4 — Compute Query Scores [$P(q|d) = P(\text{'revenue'}|d) \times P(\text{'down'}|d)$]

Score for d1:

$$P(q | d_1) = (1/8) \times (3/32) = 3/256 = \text{approx } 0.01172$$

Score for d2:

$$P(q | d_2) = (1/8) \times (1/32) = 1/256 = \text{approx } 0.00391$$

Document	$P(\text{'revenue'} d)$	$P(\text{'down'} d)$	$P(q d)$	Rank
d1 (Xyzy...revenue...down)	$1/8 = 0.125$	$3/32 = 0.09375$	$3/256 = 0.01172$	1st
d2 (Quorus...revenue...)	$1/8 = 0.125$	$1/32 = 0.03125$	$1/256 = 0.00391$	2nd

Conclusion

Final ranking: $d_1 > d_2$

d1 ranks higher because it contains 'down' while d2 does not. Even though 'down' gets a nonzero probability in d2 (thanks to smoothing), it is much smaller than in d1. Smoothing ensures d2 is not eliminated (no zero probability), but d1 correctly ranks higher. This is exactly what we want.

COMMON STUDENT MISTAKE

Mistake 1: Using $P(t|Mc)$ directly instead of $0.5 \times P(t|Mc)$ — always apply lambda! Mistake 2: Computing $P(t|d) = \text{tf}/L_d$ directly without smoothing. Mistake 3: Forgetting to compute the collection model $P(t|Mc)$ first — you need it before smoothing.

EXAM TIP

Key formulas to memorize: (1) MLE: $P_{\text{mle}}(t|d) = \text{tf}_{t,d} / L_d$. (2) Jelinek-Mercer: $P(t|d) = \lambda \times P_{\text{mle}}(t|d) + (1-\lambda) \times P(t|Mc)$. (3) Bayesian: $P(t|d) = (\text{tf}_{t,d} + \alpha \times P(t|Mc)) / (L_d + \alpha)$. (4) Final ranking: $P(q|d) = \text{product over query terms of smoothed } P(t|d)$.

Quick Reference Summary — Chapter 12 (Sections 12.1 through 12.2.2)

Section	Key Formula	Key Idea
12.1 Intro	Rank by $P(q M_d)$	Documents ranked by how likely they generate the query
12.1.1	Sum of $P(s)$ over all $s = 1$	LM = probability distribution over all strings
12.1.1 (calc)	$P(\text{string}) = \text{product of word probs} \times \text{continue/stop probs}$	Multiply all word probabilities — result is tiny but comparable
12.1.2	$P_{\text{uni}}(t_1 t_2 t_3) = P(t_1) \times P(t_2) \times P(t_3)$	IR uses unigram: independent words, no context — 3 reasons
12.1.3	$P(d)$ proportional to product of $P(t)^{tf_{t,d}}$	Query multinomial coefficient cancels; L_d is NOT fixed — L_q is fixed
12.2.1	$P(d q) = P(q d) \times P(d) / P(q) \rightarrow \text{rank by } P(q d)$	$P(q)$ and $P(d)$ drop out; rank by query likelihood
12.2.2 MLE	$P_{\text{mle}}(t d) = tf_{t,d} / L_d$	Simple count-based estimate — has zero probability problem
12.2.2 JM	$P(t d) = \lambda \times P_{\text{mle}}(t d) + (1-\lambda) \times P(t M_c)$	Linear interpolation smoothing — mixes document and collection
12.2.2 Bayes	$P(t d) = (tf_{t,d} + \alpha \times P(t M_c)) / (L_d + \alpha)$	Bayesian smoothing — α controls strength of collection prior